

OS PROJECT

Description détaillée, des fonctionnalités du projet

Fonctionnalités Détaillées du Projet `sys-watch`

Le script `sys-watch` est conçu comme un outil d'audit système et d'analyse de logs. Il est structuré autour de **trois phases** et de **sept options** obligatoires.

Phase 1 : Profilage Système (Analyse de Santé)

Objectif : Capturer l'état instantané du serveur au moment de l'exécution.

Détails techniques :

- **Commandes utilisées** : `uptime`, `free -m`, `df -h`, `ps aux`.
 - **Parsing** : Utilisation de `awk` et `grep` pour extraire uniquement les valeurs numériques (ex: *Load Average: 0.15, 0.22, 0.18*).
 - **Pourquoi c'est utile ?** Cela permet de corréliser une erreur applicative (Phase 2) avec une saturation mémoire ou CPU. Par exemple : *"L'erreur 500 est survenue exactement quand la RAM était pleine."*
-

Phase 2 : Analyse Légale (Forensique de Logs)

Objectif : Scanner les fichiers de logs du service cible pour trouver des anomalies.

Détails techniques :

- **Localisation automatique** : Le script prend en paramètre `<SERVICE_NAME>` (ex: `apache2`). Il cherche automatiquement le dossier correspondant dans `/var/log/apache2/`.
- **Recherche Regex avancée** :
 - Extraction des **adresses IP uniques** : `grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort | uniq -c`.
 - Comptage des **Erreurs HTTP 404/500** : `grep -c " 404 " error.log`.

- Filtrage par **Période** : Utilisation de `awk` pour n'afficher que les logs des 15 dernières minutes en comparant les timestamps.
- **Pourquoi c'est utile ?** Cela automatise la détection d'attaques DDoS (trop de connexions depuis une même IP) ou de bugs applicatifs récurrents.

Phase 3 : Génération de Rapport et Archivage

Objectif : Produire un fichier compressé contenant toutes les preuves collectées.

Détails techniques :

- **Contenu du rapport** : Un fichier texte `report.txt` contenant le résultat de la Phase 1 et de la Phase 2.
- **Nommage** : `AAAA-MM-JJ_HH-MM-SS_<SERVICE>_report.tar.gz` . Ce format est exigé par le cahier des charges pour le nommage des logs.
- **Compression** : Utilisation de `tar -czf` .
- **Stockage** : Les archives sont placées dans `/var/log/sys-watch/archives/` .

Explication des Options de la Ligne de Commande

Voici le détail du comportement de chaque option

Option	Nom Complet	Fonctionnalité Détaillée	Directive Respectée
<code>-h</code>	Help	Affiche un manuel complet (SYNOPSIS, DESCRIPTION, OPTIONS, EXEMPLES). C'est obligatoire et doit être appelé automatiquement en cas d'erreur de saisie.	3.2.2 & 3.2.3
<code>-s</code>	Subshell	Exécute toute la logique métier (Phases 1, 2, 3) entre parenthèses <code>()</code> . Apprentissage : Si vous modifiez une variable d'environnement (ex: <code>PATH</code>) dans le script avec <code>-s</code> , la modification est	3.2.2

Option	Nom Complet	Fonctionnalité Détaillée	Directive Respectée
		perdue à la fin. C'est idéal pour des tests isolés.	
-f	Fork	Pour les gros fichiers de logs (> 100 Mo), le script découpe le fichier en 4 morceaux. Il lance ensuite 4 processus enfants en arrière-plan (<code>&</code>) pour analyser chaque morceau en parallèle.

Apprentissage : Vous verrez plusieurs PID (Process ID) distincts avec <code>ps aux</code> . C'est la démonstration de la Charge Moyenne.	3.2.2 & 3.2.4
-t	Thread	Appelle un programme externe écrit en Langage C (<code>compress_helper</code>). Ce programme utilise la librairie <code>pthread</code> pour compresser les archives en parallèle.

Apprentissage : Vous apprendrez à compiler du C (<code>gcc -pthread</code>) et à observer la différence entre Processus Lourds (Fork) et Threads Légers (Thread) dans <code>htop</code> . C'est le test de Charge Lourde.	3.2.2 & 3.2.4
-l <rep>	Log Directory	Permet de changer l'emplacement du fichier <code>history.log</code> . Par défaut il est dans <code>/var/log/sys-watch/</code> .

Contrainte : Si vous utilisez <code>-l /tmp/</code> , vous n'avez pas besoin de <code>sudo</code> . C'est pratique pour les tests étudiants.	3.2.2
-r	Restore	Option Critique. Supprime toutes les anciennes archives et réinitialise la configuration.

 Sécurité : Le script vérifie OBLIGATOIREMENT si l'utilisateur est <code>root</code> (<code>\$EUID -eq 0</code>).	3.2.2 (Admin)

Option	Nom Complet	Fonctionnalité Détaillée	Directive Respectée
		Sinon, il refuse de s'exécuter avec un code d'erreur 103 .	
-v	Verbose	Mode debug. Affiche en temps réel la commande exacte qui est en train d'être exécutée par le shell (set -x).	Bonus

⚠ Gestion des Erreurs et Journalisation (La partie la plus notée)

C'est la partie technique la plus stricte du devoir. Voici comment elle fonctionne dans `sys-watch` :

1. La fonction `log_event` (Double Sortie)

```
# Chaque fois que le script veut dire quelque chose, il appelle cette fonction.
# Elle écrit SIMULTANÉMENT sur l'écran ET dans /var/log/sys-watch/history.log
log_event "INFOS" "Démarrage de l'analyse du service $1"
```

Format exact exigé : `AAAA-MM-JJ-HH-MM-SS : UTILISATEUR : TYPE : MESSAGE`

2. Codes de Retour Spécifiques

En cas de problème, le script ne fait pas juste `exit 1`. Il utilise des codes précis pour le débogage :

Code	Signification	Exemple de déclenchement
100	Option inconnue	<code>./sys-watch -z</code>
101	Paramètre <code><SERVICE_NAME></code> manquant	<code>./sys-watch -s</code> (sans dire quel service)
102	Dossier de logs introuvable	<code>./sys-watch mysql</code> (alors que MySQL n'est pas installé)
103	Privilèges insuffisants	<code>./sys-watch -r</code> (exécuté sans <code>sudo</code>)

3. Affichage Automatique de l'Aide

Dès qu'un code d'erreur (100, 101, 102, 103) est déclenché, le script appelle **automatiquement** la fonction `display_help`. L'utilisateur voit immédiatement comment corriger sa saisie, sans avoir à retaper `-h`. C'est une exigence explicite du document (Section 3.2.3).

Scénarios de Test

Le projet est prévu pour que vous puissiez réaliser les trois démonstrations demandées facilement :

1. **Test Léger (s)** : `./sys-watch -s cron`
 - *Résultat* : Le rapport se génère en moins d'une seconde. Vous montrez que le script tourne dans un sous-shell via `echo $$` (le PID change).
 2. **Test Moyen (f)** : `./sys-watch -f nginx` (avec un fichier `access.log` de 200 Mo)
 - *Résultat* : Vous lancez `htop` en parallèle. Vous verrez **4 processus** `sys-watch` qui tournent en même temps (Fork). L'analyse est 4x plus rapide.
 3. **Test Lourd (t)** : `./sys-watch -t apache2` (avec un dossier de logs de 2 Go)
 - *Résultat* : Le helper C est appelé. Dans `htop` (vue en arbre `F5`), vous montrez que la compression utilise **plusieurs threads** mais un **seul PID**. C'est la preuve de la maîtrise de la différence Processus/Thread.
-

Résumé pour l'Apprentissage

En réalisant ce projet `sys-watch`, vous allez concrètement maîtriser :

1. **Les Redirections Avancées** : `tee`, `flock` (verrouillage de fichier).
2. **La Programmation Shell Robuste** : `getopts`, `shift`, `case`, `trap`.
3. **L'Interfaçage Bash/C** : Compilation conditionnelle et passage d'arguments.
4. **L'Administration Système** : Gestion des permissions (`/var/log`), parsing de `/proc`.
5. **La Gestion de Projet** : Respect strict d'un cahier des charges professionnel.

La pratie Technique :

Here is a comprehensive, unified project description that merges the best aspects of system monitoring, log analysis, and environment automation into a single, assignment-perfect solution. This script is designed to meet **every single directive** in your ENSET document while teaching you the deep fundamentals of Unix/Linux process management.

Project Title: `sys-watch`

Subtitle: *Intelligent System Profiler & Automated Report Generator*

1. Context and Need (Real-World Justification)

System administrators and developers frequently face two disconnected but critical tasks:

1. **Reactive Analysis:** Parsing massive service logs (Apache/Nginx) to find the root cause of a crash or performance drop.
2. **Proactive Monitoring:** Checking CPU, memory, and disk health at the moment of failure to correlate logs with resource exhaustion.

Currently, a junior admin must run `top`, `df -h`, `grep "ERROR" /var/log/app.log`, and then manually stitch this data together in an email. This is slow, prone to human error, and unrepeatable.

The Need: A single automated tool that captures a **System State Snapshot** AND **Analyzes Critical Logs** in one atomic operation, with different execution modes for lightweight checks versus heavy forensic deep-dives.

2. Mandatory Parameter

The script requires **one mandatory argument**: the **Target Service Name** (e.g., `apache2`, `nginx`, `mysql`).

- The script will use this name to locate the appropriate log directory (`/var/log/<service>`) and check the process list for that service.

3. Command Syntax & Options (Full Compliance)

```
sys-watch [OPTIONS] <SERVICE_NAME>
```

Option	Functionality	Directive Met
<code>-h</code>	Displays detailed help with examples.	Help Option
<code>-s</code>	Executes the main task in a subshell . Useful for isolating environment variables during light tests.	Subshell Execution
<code>-f</code>	Executes the log analysis and system checks in parallel using background processes (Fork) .	Fork Execution
<code>-t</code>	Executes the log compression task using a C multithreading helper for maximum CPU efficiency.	Thread Execution
<code>-l <dir></code>	Overrides the default log path (<code>/var/log/sys-watch/</code>) with a custom directory.	Logging Directory
<code>-r</code>	Admin Only. Resets the entire monitoring environment: deletes all historical archives and restores default configs.	Restore / Admin Privilege
<code>-v</code>	Extra Option. Verbose mode. Shows real-time <code>strace</code> -like output of internal commands.	Additional Option

4. Core Functionality Workflow

The script will perform the following **Three-Phase Analysis** for the given

`<SERVICE_NAME>` :

1. Phase 1: System Profiling (Light Load)

- Captures: CPU Load Average, Memory Usage (`free -m`), Disk I/O, and Top 5 Processes.
- Uses `/proc` **filesystem** parsing and `awk` to format data.

2. Phase 2: Log Forensics (Medium/Heavy Load)

- Locates the log file: `/var/log/<SERVICE_NAME>/error.log` or `access.log`.
- **Parses with Regex:** Extracts unique IP addresses (potential attackers), counts 404/500 errors, and isolates entries from the last 15 minutes.

- **Heavy Load Trigger:** If the log file is larger than 100MB, the script automatically switches to a "chunked processing" mode using **Fork** or **Thread** options.

3. Phase 3: Reporting & Archiving

- Generates a compressed report: `YYYY-MM-DD_HH-MM-SS_<SERVICE_NAME>_report.tar.gz`.
- Archives the report in `/var/log/sys-watch/archives/`.

5. Implementation Details (How It Teaches You Unix/Linux)

A. Strict Logging Function (Meets `history.log` Spec)

This function will be the heart of the script. It satisfies the requirement for **simultaneous Terminal + Logfile output** with exact timestamp format.

```
LOG_BASE="/var/log/sys-watch" # Default, changeable with -l

log_event() {
    local type="$1" msg="$2"
    local timestamp=$(date '+%Y-%m-%d-%H-%M-%S')
    local entry="$timestamp : $USER : $type : $msg"

    # 1. Terminal Output
    echo "$msg"

    # 2. File Output with Lock (critical for -f fork mode to
    avoid log corruption)
    flock "$LOG_BASE/history.log" -c "echo \"$entry\" >>
    \"$LOG_BASE/history.log\""
}
```

B. Advanced Error Handling & Auto-Help

The assignment requires that any error triggers an **automatic display of the help menu**. We will create an `error_exit` trap:


```

error_exit() {
    local code=$1 msg=$2
    log_event "ERROR" "$msg (Exit Code: $code)"
    display_help # <-- AUTO-HELP DISPLAY (Requirement 3.2.3)
    exit $code
}

# Custom Exit Codes (Requirement 3.2.3)
# 100: Invalid Option
# 101: Service Name Missing
# 102: Service Log Directory Not Found
# 103: Insufficient Permissions for -r (Restore)

```

C. Execution Modes: Subshell, Fork, Thread (Requirement 3.2.2 & 3.2.4)

This is the **most critical learning component** of the project. The script will use a `case` statement to handle how the main `analyze()` function is called.

- **s Subshell:** `( analyze "$service" )`
 - **Learning Outcome:** Demonstrates variable scoping. Changes made to `PATH` inside the subshell do not affect the parent script.
- **f Fork (Background Process):** `analyze_chunk "$log_part" &`
 - **Learning Outcome:** Demonstrates process management (`jobs` , `wait` , `!`). For large logs, the script splits the file and forks multiple analyzers simultaneously. This is the **Medium Load Test**.
- **t Thread (C Helper):**
 - **The Challenge:** Bash has no threads.
 - **The Solution (Learning Payoff):** You will write a **20-line C program** (`compress_thread.c`) that uses `pthread` to run `gzip` on the final report file in parallel chunks.
 - **Heavy Load Test:** Using `t` on a 2GB log file should complete compression faster than using `f` due to shared memory vs process overhead. You will

document this performance difference in your PDF report.

D. Restore Option **r** (Admin Privilege Check)

```
if [ "$1" == "-r" ]; then
    if [ "$EUID" -ne 0 ]; then
        error_exit 103 "Restore mode requires root privilege
s. Use sudo."
    fi
    rm -rf /var/log/sys-watch/archives/*
    log_event "INFOS" "Environment reset by administrator $US
ER."
    exit 0
fi
```

6. Testing Scenarios (Light, Medium, Heavy)

Your PDF report will include screenshots of these exact tests:

1. Light Load (Subshell **s**):

- Command: `./sys-watch -s cron`
- **Expected:** Script runs quickly inside `( )`. The `history.log` shows the user session isolated.

2. Medium Load (Fork **f**):

- Command: `./sys-watch -f nginx` (on a 50MB log file)
- **Expected:** The script splits the file into 4 parts. `ps aux | grep sys-watch` shows **4 child processes** running simultaneously.

3. Heavy Load (Thread **t**):

- Command: `./sys-watch -t apache2` (on a 500MB log file)
- **Expected:** The C helper program spawns multiple threads. You demonstrate using `htop` (Tree View) that **Threads** share the same PID but have different TIDs, whereas Forks have different PIDs.

7. Deliverables Structure

Your `TeamID-devoir-shell.zip` will contain:

File	Content Highlights
<code>sys-watch</code>	The main Bash script. Named logically as per directive.
<code>compress_helper.c</code>	The C program for <code>-t</code> option. Compiled via <code>gcc -pthread</code> .
<code>TeamID-devoir-shell.pdf</code>	Extended documentation with code snippets, regex explanation (<code>grep -E</code>), and performance comparison graph (Fork vs Thread time).
<code>TeamID-devoir-shell.pptx</code>	One Slide: Title, Team Members, Bullet points: <i>The Problem, The Solution, Key Feature (Fork/Thread Demo)</i> .

This unified project, `sys-watch`, demonstrates **every** requirement in your assignment while giving you a genuine, practical understanding of how professional Linux tools like `logrotate` and `top` are built under the hood.

Imohim

Imohim

Imohim

Imohim

Imohim

Imohim

Imohim

Imohim

Imohim